

Software Architecture Document V2

Project Information

Project name: PromoCatch - Campaign and Deal Tracking System

Course context: Software Architecture Project

Phase: Phase 2 - Architecture Completion and Full Implementation

Deadline: 24.05.2026

Architecture style: Layered Architecture with REST API

Technology stack:

- Backend: Python, FastAPI, SQLAlchemy
- Database: SQLite
- Frontend: HTML, CSS, Bootstrap, Vanilla JavaScript
- Deployment: Docker, Docker Compose, Uvicorn

Phase 2 Step 1: Extended SAD (Version 2)

1) Completed Use Case View

Actors:

- User: tracks, searches, creates, edits, and deletes campaigns
- System: validates data, persists campaign records, and serves API responses

Completed use cases:

1. View Campaign List
2. Search Campaigns
3. Filter Campaigns by Platform and Discount
4. Create Campaign
5. Update Campaign
6. Delete Campaign
7. View Campaign Detail

Use case diagram:

```
@startuml
left to right direction
actor "User" as User
```

```

rectangle "PromoCatch Campaign Tracking System" {
    usecase "View Campaign List" as UC1
    usecase "Search Campaigns" as UC2
    usecase "Filter by Platform/Discount" as UC3
    usecase "Create Campaign" as UC4
    usecase "Update Campaign" as UC5
    usecase "Delete Campaign" as UC6
    usecase "View Campaign Detail" as UC7
}

User --> UC1
User --> UC2
User --> UC3
User --> UC4
User --> UC5
User --> UC6
User --> UC7
@enduml

```

Primary flow (search and create):

1. User opens PromoCatch.
2. Frontend requests GET /campaigns with optional query parameters.
3. API returns filtered campaign list.
4. User fills form and submits campaign.
5. Frontend sends POST /campaigns.
6. API validates payload and stores campaign.
7. Frontend refreshes list and displays updated state.

Alternative flow (update/delete):

1. User opens a campaign in edit mode.
2. Frontend sends PUT /campaigns/{campaign_id} for updates.
3. API validates and updates the campaign.
4. User can remove campaign with DELETE /campaigns/{campaign_id}.
5. API removes row and returns success response.

2) Completed Logical View

Layered logical structure:

- Presentation Layer: static/index.html
- Control Layer: main.py, routers.py
- Domain Layer: schemas.py, services.py
- Resource Layer: database.py, models.py, repository.py

Extended component diagram:

```
@startuml
```

```

package "Presentation Layer" {
  [index.html]
  [Fetch API Client]
}

package "Control Layer" {
  [FastAPI App]
  [Campaign Router]
}

package "Domain Layer" {
  [Campaign Service]
  [Validation Schemas]
}

package "Resource Layer" {
  [Campaign Repository]
  [Campaign ORM Model]
  database "SQLite Database" as DB
}

[index.html] --> [Fetch API Client] : user actions
[Fetch API Client] --> [Campaign Router] : HTTP GET/POST/PUT/DELETE
[FastAPI App] --> [Campaign Router] : include_router
[Campaign Router] --> [Campaign Service] : orchestration
[Campaign Service] --> [Validation Schemas] : validation/normalization
[Campaign Service] --> [Campaign Repository] : persistence operations
[Campaign Repository] --> [Campaign ORM Model] : ORM mapping
[Campaign Repository] --> DB : SQL operations
@enduml

```

Domain model:

```

@startuml
class Campaign {
  +int id
  +string title
  +string platform
  +string description
  +int discount_rate
}

class CampaignCreate {
  +string title
  +string platform
  +string description
  +int discount_rate
}

class CampaignUpdate {
  +string title
  +string platform
  +string description
  +int discount_rate
}

class CampaignRead {
  +int id
  +string title
  +string platform
  +string description
  +int discount_rate
}

CampaignCreate --> Campaign
CampaignUpdate --> Campaign

```

```
Campaign --> CampaignRead
@enduml
```

3) Completed Process View

Runtime behavior

- Browser and backend communicate synchronously over HTTP.
- Router orchestrates requests and delegates business processing to services.
- Services normalize payloads and enforce domain rules before repository calls.
- Repository performs all SQLAlchemy database access operations.

Sequence - update campaign flow

```
@startuml
actor User
participant "Browser UI" as UI
participant "Router" as Router
participant "Service" as Service
participant "Repository" as Repo
database "SQLite" as DB

User -> UI : Submit edit form
UI -> Router : PUT /campaigns/{id}
Router -> Service : update_campaign(id, data)
Service -> Repo : get_campaign_by_id(id)
Repo -> DB : SELECT campaign
DB --> Repo : Campaign row
Repo --> Service : Campaign entity
Service -> Repo : update_campaign(entity, data)
Repo -> DB : UPDATE campaign
DB --> Repo : Updated row
Repo --> Service : Updated campaign
Service --> Router : CampaignRead
Router --> UI : 200 OK
UI --> User : Render updated campaign
@enduml
```

Sequence - delete campaign flow

```
@startuml
actor User
participant "Browser UI" as UI
participant "Router" as Router
participant "Service" as Service
participant "Repository" as Repo
database "SQLite" as DB

User -> UI : Click delete
UI -> Router : DELETE /campaigns/{id}
Router -> Service : delete_campaign(id)
Service -> Repo : get_campaign_by_id(id)
Repo -> DB : SELECT campaign
DB --> Repo : Campaign row
Service -> Repo : delete_campaign(entity)
Repo -> DB : DELETE campaign
DB --> Repo : delete success
Service --> Router : result
Router --> UI : 204 No Content
UI --> User : Refresh list
```

4) Development View

Module organization:

```
PromoCatch/  
  main.py  
  routers.py  
  services.py  
  schemas.py  
  repository.py  
  models.py  
  database.py  
  static/  
    index.html  
  README.md  
  SAD_Phase1.md  
  SAD_Phase2.md  
  requirements.txt  
  Dockerfile  
  docker-compose.yml
```

Module responsibilities:

- `main.py`: application startup, table creation, static UI serving
- `routers.py`: REST endpoint definitions and HTTP contract
- `services.py`: domain logic, normalization, and workflow handling
- `repository.py`: database queries and persistence
- `schemas.py`: request/response contracts with validation
- `static/index.html`: complete UI behavior for full campaign management

Development view UML component diagram:

```
@startuml  
package "PromoCatch Codebase" {  
  component "main.py\n(App Bootstrap)" as Main  
  component "routers.py\n(API Controller)" as Router  
  component "services.py\n(Domain Services)" as Service  
  component "repository.py\n(Data Access)" as Repo  
  component "models.py\n(SQLAlchemy Model)" as Model  
  component "schemas.py\n(Pydantic Contracts)" as Schema  
  component "database.py\n(DB Session/Engine)" as DBConfig  
  component "static/index.html\n(UI + Fetch Client)" as UI  
}
```

```
database "SQLite (promocatch.db)" as DB
```

```
UI --> Router : HTTP REST calls  
Main --> Router : include_router()  
Main --> DBConfig : initialize engine/session  
Router --> Service : delegate request workflow  
Router --> Schema : request/response typing  
Service --> Repo : business operations  
Service --> Schema : validation + normalization  
Repo --> Model : ORM mapping  
Repo --> DBConfig : session usage  
DBConfig --> DB : connection
```

Repo --> DB : SQL operations
@enduml

5) Deployment View

Target environment

- Container 1 (app): FastAPI + Uvicorn runtime
- Persistent storage: SQLite file mounted as a volume
- Client access: Browser via `http://localhost:8000`

Deployment diagram:

```
@startuml
node "Developer/Production Host" {
  node "Docker Engine" {
    node "promocatch-app container" {
      artifact "FastAPI + Uvicorn"
      artifact "PromoCatch source code"
    }
    database "promocatch.db (mounted volume)"
  }
  artifact "Web Browser"
}

"Web Browser" --> "FastAPI + Uvicorn" : HTTP 8000
"FastAPI + Uvicorn" --> "promocatch.db (mounted volume)" : SQLAlchemy (SQLite)
@enduml
```

Deployment strategy:

1. Build image from Dockerfile.
2. Start service with `docker compose up --build`.
3. Access UI at `http://localhost:8000`.
4. Persist `promocatch.db` through host volume mapping.

Phase 2 Step 2: Implementation (Version 2)

Completed implementation in this version:

- Full frontend flows: list, search, filter (including min-max discount range), create, update, and delete campaign.
- Full backend REST API flows: GET, GET by id, POST, PUT, and DELETE.
- Filter-based querying using search text, platform, and discount criteria.
- Integer-only discount policy (`discount_rate` is 1-100) with fast UI controls (preset buttons and slider).
- Layered architecture preserved with clear separation of responsibilities.
- Deployment-ready container setup using `Dockerfile` and `docker-compose.yml`.

Team Member Contributions

Team Member	Contribution
Cemil Tekin	Extended SAD V2 architecture and implementation alignment, backend/full-stack integration
Kaan Kesen	SAD structure review, diagrams and documentation organization
Ömer Tarık Çandır	Frontend implementation completion and API integration testing